

Assignment 01

Digital RTL Design

Combinational Circuit Design

Marwan Abdelfattah

Contents

1	Question 1	2
1.1	Problem Statement	2
1.2	RTL Design	2
1.3	Testbench	3
1.4	Simulation Waveform	4
2	Question 2	5
2.1	Problem Statement	5
2.2	RTL Design	5
2.3	Testbench	6
2.4	Simulation Waveform	7
3	Question 3	8
3.1	Problem Statement	8
3.2	RTL Design	8
3.3	Testbench	8
3.4	Simulation Waveform	9
4	Question 4	10
4.1	Problem Statement	10
4.2	RTL Design	10
4.3	Testbench	10
4.4	Simulation Waveform	11
5	Question 5	12
5.1	Problem Statement	12
5.2	RTL Design	12
6	Question 6	13
6.1	Problem Statement	13
6.2	4-Bit ALU with 7-Segment Display	13
6.3	RTL Design	13
6.4	Testbench	14
6.5	Simulation Waveform	15
7	Conclusion	15

1 Question 1

1.1 Problem Statement

A 4-bit binary number is represented as

$$A_3A_2A_1A_0$$

where A_3 , A_2 , A_1 , and A_0 represent the individual bits, with A_0 being the **least significant bit (LSB)**.

Design a logic circuit using **Verilog** that produces a **HIGH** output whenever the 4-bit binary input satisfies the following condition:

$$0010 < A_3A_2A_1A_0 < 1000$$

Therefore, the output should be HIGH for binary input values ranging from 0011 to 0111 (decimal values 3 through 7), and LOW for all other input values.

1.2 RTL Design

The following Verilog RTL implementation was developed according to the requirements of Question 1.

```
1 module ct1 (  
2     A, B, C, D, E, F, sel, out, out_bar  
3 );  
4  
5     input A, B, C, D, E, F, sel;  
6     output out, out_bar;  
7     wire and1_out, xnor1_out, mux_out;  
8  
9     assign and1_out = A & B & C;  
10    assign xnor1_out = ~(D ^ E ^ F);  
11  
12    assign out = (sel == 0) ? and1_out : xnor1_out;  
13    assign out_bar = ~out;  
14  
15  
16 endmodule
```

1.3 Testbench

The following testbench was used to verify the functionality of the RTL design using different input combinations.

```

1 module ct1_tb ();
2
3 reg a_tb, b_tb, c_tb, d_tb, e_tb, f_tb, sel_tb;
4 reg out_expected, out_bar_expected;
5 wire out_dut, out_bar_dut;
6
7 ct1 DUT (.A(a_tb), .B(b_tb), .C(c_tb), .D(d_tb), .E(e_tb), .F(f_tb), .sel(
8     sel_tb), .out(out_dut), .out_bar(out_bar_dut));
9
10 integer i;
11
12 initial begin
13     for(i=0; i<20; i=i+1) begin
14         a_tb = $random % 2;
15         b_tb = $random % 2;
16         c_tb = $random % 2;
17         d_tb = $random % 2;
18         e_tb = $random % 2;
19         f_tb = $random % 2;
20         sel_tb = $random % 2;
21         out_expected = (sel_tb == 1)? (~(d_tb ^ e_tb ^ f_tb)): (a_tb & b_tb &
22             c_tb);
23         out_bar_expected = ~out_expected;
24         #10;
25
26         if((out_dut == out_expected) && (out_bar_dut == out_bar_expected)) begin
27             $display("Circuit Working Correctly");
28         end
29         else begin
30             $display("Circuit Failed!!!");
31             $stop;
32         end
33     end
34 end
35
36 $stop;
37 end
38
39 initial begin
40     $monitor("A= %d, B= %d, C= %d, D= %d, E= %d, F= %d, Sel = %d, out= %d,
41         out_bar=%d",
42         a_tb, b_tb, c_tb, d_tb, e_tb, f_tb, sel_tb, out_dut,
43         out_bar_dut);
44 end
45
46 endmodule

```

1.4 Simulation Waveform

The design was simulated using QuestaSim with different input combinations to verify its functionality.

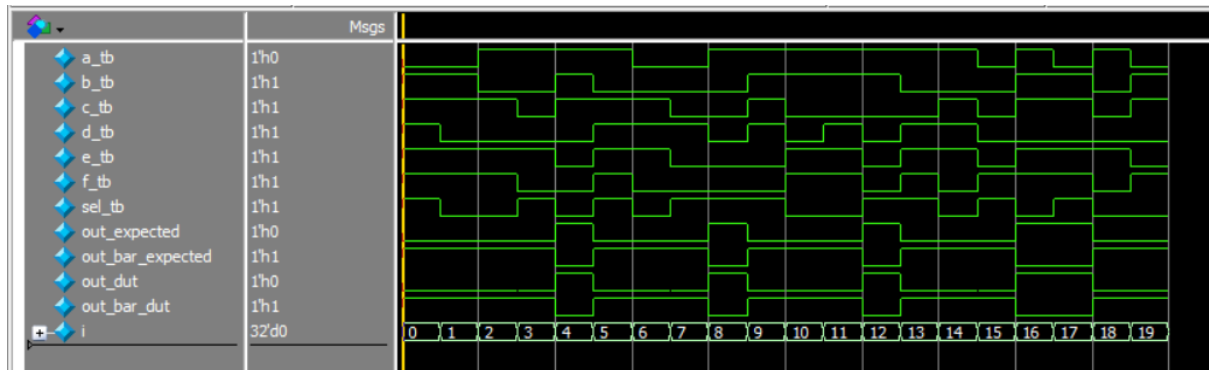


Figure 1: Simulation waveform for Question 1

```
# Loading work.ct1(fast)
# A= 0, B= 1, C= 1, D= 1, E= 1, F= 1, Sel = 1, out= 0, out_bar=1
# Circuit Working Correctly
# A= 0, B= 1, C= 1, D= 0, E= 1, F= 1, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 0, C= 1, D= 0, E= 1, F= 1, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 0, C= 0, D= 0, E= 1, F= 0, Sel = 1, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 1, C= 1, D= 0, E= 0, F= 0, Sel = 0, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 0, C= 1, D= 1, E= 1, F= 1, Sel = 1, out= 0, out_bar=1
# Circuit Working Correctly
# A= 0, B= 0, C= 1, D= 1, E= 1, F= 0, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# A= 0, B= 0, C= 0, D= 1, E= 0, F= 0, Sel = 1, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 0, C= 0, D= 0, E= 0, F= 0, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 1, C= 1, D= 1, E= 0, F= 0, Sel = 1, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 1, C= 0, D= 0, E= 1, F= 1, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 1, C= 0, D= 1, E= 1, F= 1, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 1, C= 0, D= 0, E= 0, F= 0, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 0, C= 0, D= 1, E= 1, F= 1, Sel = 1, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 0, C= 1, D= 1, E= 1, F= 0, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# A= 0, B= 0, C= 0, D= 0, E= 0, F= 1, Sel = 1, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 1, C= 1, D= 0, E= 1, F= 1, Sel = 0, out= 1, out_bar=0
# Circuit Working Correctly
# A= 0, B= 1, C= 1, D= 0, E= 1, F= 1, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 0, C= 0, D= 0, E= 1, F= 0, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# A= 0, B= 1, C= 1, D= 0, E= 0, F= 1, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# ** Note: $stop      : ctl_tb.v(45)
#   Time: 200 ns   Iteration: 0   Instance: /ctl_tb
# Break in Module ctl_tb at ctl_tb.v line 45
```

Figure 2: Transcript for Question 1

2 Question 2

2.1 Problem Statement

The design consists of **5 inputs** and **2 outputs**.

- **Inputs**

- D: 3-bit input
- A, B, C, Sel: 1-bit inputs

- **Outputs**

- Out, outbar: 1-bit outputs

The design should be implemented using a **behavioral coding style** according to the following specifications.

2.2 RTL Design

The following Verilog RTL implementation was developed according to the requirements of Question 1.

```
1 module ct2 (A,B,C,D,SEL, out, out_bar);
2
3 input A, B, C, SEL;
4 input [2:0] D;
5 output reg out, out_bar;
6 reg and2_out, or1_out, xnor1_out;
7
8
9 always @(*) begin
10     and2_out = D[0] & D[1];
11     or1_out = D[2] | and2_out;
12     xnor1_out = ~(A ^ B ^ C);
13
14     if(SEL)begin
15         out = xnor1_out;
16     end
17     else begin
18         out = or1_out;
19     end
20
21 out_bar = ~out;
22 end
23
24
25
26 endmodule
```

2.3 Testbench

The following testbench was used to verify the functionality of the RTL design using different input combinations.

```

1 module ct2_tb();
2
3 reg a_tb, b_tb, c_tb, sel_tb;
4 reg [2:0] d_tb;
5 reg out_expected, out_bar_expected;
6 wire out_dut, out_bar_dut;
7
8 ct2 DUT (
9     .A(a_tb),
10    .B(b_tb),
11    .C(c_tb),
12    .D(d_tb),
13    .SEL(sel_tb),
14    .out(out_dut),
15    .out_bar(out_bar_dut)
16 );
17
18 integer i;
19
20 initial begin
21
22     for(i=0; i<20; i=i+1) begin
23         a_tb = $random % 2;
24         b_tb = $random % 2;
25         c_tb = $random % 2;
26         d_tb = $random % 8;
27         sel_tb = $random % 2;
28         out_expected = (sel_tb == 1)? (~(a_tb ^ b_tb ^ c_tb)): ((d_tb[0] &
29         d_tb[1]) | d_tb[2]);
30         out_bar_expected = ~out_expected;
31         #10;
32         if((out_dut == out_expected) && (out_bar_dut == out_bar_expected)) begin
33             $display("Circuit Working Correctly");
34         end
35         else begin
36             $display("Circuit Failed!!!");
37             $stop;
38         end
39
40
41     end
42     $display("\nAll tests passed!");
43     $stop;
44 end
45
46 initial begin

```

```

47     $monitor("A= %d, B= %d, C= %d, D= %d, Sel = %d, out= %d, out_bar=%d",
48             a_tb, b_tb, c_tb, d_tb, sel_tb, out_dut, out_bar_dut);
49 end
50
51
52
53
54 endmodule

```

2.4 Simulation Waveform

The design was simulated using QuestaSim with different input combinations to verify its functionality.

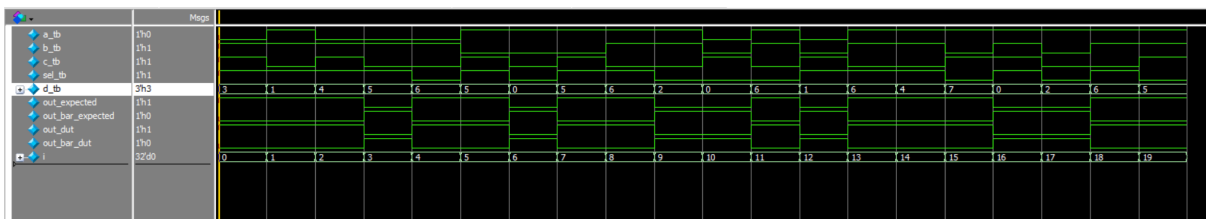


Figure 3: Simulation waveform for Question 2

```

# Loading work.ct2_tb(fast)
# Loading work.ct2(fast)
# A= 0, B= 1, C= 1, D= 3, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 1, C= 0, D= 1, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 0, B= 1, C= 1, D= 4, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 0, B= 1, C= 0, D= 5, Sel = 1, out= 0, out_bar=1
# Circuit Working Correctly
# A= 0, B= 1, C= 0, D= 6, Sel = 0, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 0, C= 1, D= 5, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 0, C= 0, D= 0, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 0, C= 1, D= 5, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 1, C= 0, D= 6, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 1, C= 0, D= 2, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# A= 0, B= 0, C= 1, D= 0, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 1, C= 0, D= 6, Sel = 0, out= 1, out_bar=0
# Circuit Working Correctly
# A= 0, B= 0, C= 1, D= 1, Sel = 1, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 1, C= 0, D= 6, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 1, C= 0, D= 4, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 0, C= 1, D= 7, Sel = 0, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 1, C= 1, D= 0, Sel = 1, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 0, C= 0, D= 2, Sel = 0, out= 0, out_bar=1
# Circuit Working Correctly
# A= 1, B= 1, C= 0, D= 6, Sel = 1, out= 1, out_bar=0
# Circuit Working Correctly
# A= 1, B= 1, C= 1, D= 5, Sel = 0, out= 1, out_bar=0
# Circuit Working Correctly
#
# All tests passed!
# ** Note: Satop... ct2 tb v(13)

```

Figure 4: Transcript for Question 2

3 Question 3

3.1 Problem Statement

Implement a **4-bit adder** using the addition operator (+) and an **assign** statement.

- **Inputs**

- A: 4-bit input
- B: 4-bit input

- **Output**

- C: 4-bit output representing the sum of A and B

The summation of A and B is assigned to the output C, while the carry-out bit is ignored.

3.2 RTL Design

The following Verilog RTL implementation was developed according to the requirements of Question 1.

```
1 module adder (A, B, C);  
2     input [3:0] A, B;  
3     output [3:0] C;  
4     assign C = A + B;  
5 endmodule
```

3.3 Testbench

The following testbench was used to verify the functionality of the RTL design using different input combinations.

```
1 module adder_tb ();  
2     reg [3:0] A, B;  
3     reg [3:0] c_expected;  
4     wire [3:0] C;  
5     integer i;  
6  
7     adder DUT(.);  
8  
9     initial begin  
10         for(i=0;i<20;i=i+1)begin  
11             A= $random % 16;  
12             B= $random % 16;  
13             c_expected = A + B;  
14             #10  
15             if (c_expected == C) begin  
16                 $display(" Correct :) ");  
17             end else begin  
18                 $display("Error!!!!!!!!!!!!");  
19             end  
20         end  
21     end
```

```

19         $stop;
20     end
21 end
22 $display("\nAll Tests Passed");
23 $stop;
24 end
25
26 initial begin
27     $monitor("Time=%0t | A=%d, B=%d, C=%d, Expected=%d",
28             $time, A, B, C, c_expected);
29 end
30 endmodule

```

3.4 Simulation Waveform

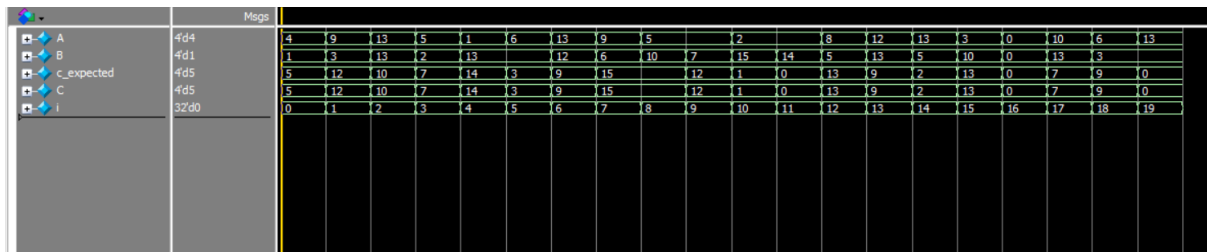


Figure 5: Simulation waveform for Question 3

```

# Time=0 | A= 4, B= 1, C= 5, Expected= 5
# Correct :)
# Time=10 | A= 9, B= 3, C=12, Expected=12
# Correct :)
# Time=20 | A=13, B=13, C=10, Expected=10
# Correct :)
# Time=30 | A= 5, B= 2, C= 7, Expected= 7
# Correct :)
# Time=40 | A= 1, B=13, C=14, Expected=14
# Correct :)
# Time=50 | A= 6, B=13, C= 3, Expected= 3
# Correct :)
# Time=60 | A=13, B=12, C= 9, Expected= 9
# Correct :)
# Time=70 | A= 9, B= 6, C=15, Expected=15
# Correct :)
# Time=80 | A= 5, B=10, C=15, Expected=15
# Correct :)
# Time=90 | A= 5, B= 7, C=12, Expected=12
# Correct :)
# Time=100 | A= 2, B=15, C= 1, Expected= 1
# Correct :)
# Time=110 | A= 2, B=14, C= 0, Expected= 0
# Correct :)
# Time=120 | A= 8, B= 5, C=13, Expected=13
# Correct :)
# Time=130 | A=12, B=13, C= 9, Expected= 9
# Correct :)
# Time=140 | A=13, B= 5, C= 2, Expected= 2
# Correct :)
# Time=150 | A= 3, B=10, C=13, Expected=13
# Correct :)
# Time=160 | A= 0, B= 0, C= 0, Expected= 0
# Correct :)
# Time=170 | A=10, B=13, C= 7, Expected= 7
# Correct :)
# Time=180 | A= 6, B= 3, C= 9, Expected= 9
# Correct :)
# Time=190 | A=13, B= 3, C= 0, Expected= 0
# Correct :)
#
# All Tests Passed
** Note: $stop      : adder_tb.v(25)
#   Time: 200 ns   Iteration: 0   Instance: /adder_tb

```

Figure 6: Transcript for Question 3

4 Question 4

4.1 Problem Statement

Implement a **2-to-4 decoder** using the **conditional operator**. A logic decoder has n input lines and 2^n output lines, where each output line corresponds to a unique combination of the input values.

- **Input**

- A: 2-bit input

- **Output**

- D: 4-bit output

The following format can be used for the conditional operator:

```
assign <output_signal> = <condition1> ? <value1> :  
                        <condition2> ? <value2> :  
                        <default_value>;
```

4.2 RTL Design

The following Verilog RTL implementation was developed according to the requirements of Question 4.

```
1 module decoder (A, D);  
2     input [1:0] A;  
3     output [3:0] D;  
4  
5     assign D = (A==0)? 4'b0001: (A==1)? 4'b0010 : (A==2)? 4'b0100 : 4'b1000 ;  
6  
7 endmodule
```

4.3 Testbench

The following testbench was used to verify the functionality of the RTL design using different input combinations.

```
1  
2 module decoder_tb;  
3     reg [1:0] A;  
4     wire [3:0] D;  
5     integer i;  
6  
7     // Instantiate the design under test  
8     decoder DUT (.A(A), .D(D));  
9  
10    initial begin  
11  
12        $display(" A | D ");
```

```

13     $display("----+-----");
14
15     for (i = 0; i < 4; i = i + 1) begin
16         A = i;
17         #10;
18         $display(" %b | %b ", A, D);
19     end
20
21     $stop;
22 end
23 endmodule

```

4.4 Simulation Waveform

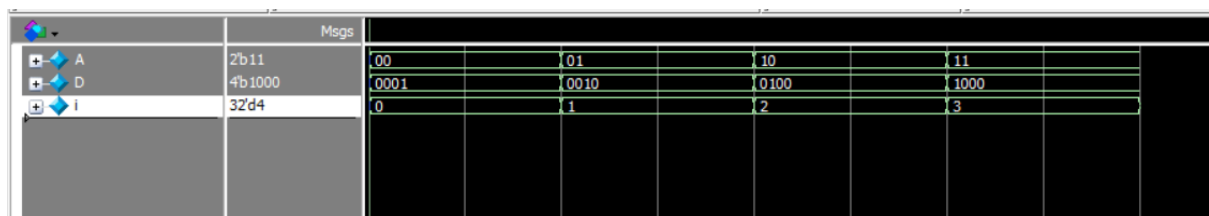


Figure 7: Simulation waveform for Question 4

```

Loading Modelsim
A | D
----+-----
00 | 0001
01 | 0010
10 | 0100
11 | 1000
** Note: $stop
Time: 40 ns :

```

Figure 8: Transcript for Question 4

5 Question 5

5.1 Problem Statement

Implement an **even parity generator** using an **assign** statement. The design uses a **reduction operator** to calculate the even parity bit from the input bus.

- **Input**
 - A: 8-bit input
- **Output**
 - out_with_parity: 9-bit output

The calculated parity bit is inserted as the **Most significant bit (MSB)** of the output bus, while the remaining 8 bits correspond to the input A.

Hint: Use the concatenation operator to combine the input A with the calculated parity bit.

5.2 RTL Design

The following Verilog RTL implementation was developed according to the requirements of Question 5.

```
1 module parity_check (A,out_with_parity);
2
3     input  [7:0] A;
4     output [8:0] out_with_parity;
5
6     assign out_with_parity = {A,^A};
7
8 endmodule
```

6 Question 6

6.1 Problem Statement

6.2 4-Bit ALU with 7-Segment Display

In this design, a **4-bit ALU** is connected to a **7-segment display** so that the result of the selected ALU operation is displayed as a hexadecimal digit from 0 to F.

- **Inputs**

- A: 4-bit input
- B: 4-bit input
- Opcode: 2-bit input used to select the ALU operation
- Enable: 1-bit input used to enable or disable the display

- **Outputs**

- a, b, c, d, e, f, g: 7-segment display control signals

The design should use the **ALU module from the previous lab** with the parameter $N = 4$. The ALU module should be instantiated inside the top-level design.

The ALU takes the inputs A, B, and Opcode and generates a 4-bit result:

Result[3 : 0]

The generated result is then passed to a **7-segment decoder**. The decoder converts the 4-bit ALU result into seven control signals (a–g) to display the corresponding hexadecimal digit.

When **Enable** is active, the 7-segment display shows the ALU result. When **Enable** is inactive, the display is disabled.

6.3 RTL Design

The following Verilog RTL implementation was developed according to the requirements of Question 5.

```

1 module segment_LED_Display (A,B,opcode, enable, a, b, c, d, e, f, g);
2
3   input  [3:0] A,B;
4   input  [1:0] opcode;
5   input  enable;
6   output reg a,b,c,d,e,f,g;
7
8   wire [3:0] alu_out;
9   reg [6:0] display_out;
10
11   Nbit_ALU #(4) alu1(.in0(A),.in1(B),.opcode(opcode),.out(alu_out));
12
13   always @(*) begin
14       if(!enable) begin

```

```

15     display_out = 7'b0000000; // All segments off
16 end else begin
17     case (alu_out)
18         4'b0000: display_out = 7'b1111110; // 0
19         4'b0001: display_out = 7'b0110000; // 1
20         4'b0010: display_out = 7'b1101101; // 2
21         4'b0011: display_out = 7'b1111001; // 3
22         4'b0100: display_out = 7'b0110011; // 4
23         4'b0101: display_out = 7'b1011011; // 5
24         4'b0110: display_out = 7'b1011111; // 6
25         4'b0111: display_out = 7'b1110000; // 7
26         4'b1000: display_out = 7'b1111111; // 8
27         4'b1001: display_out = 7'b1111011; // 9
28         4'b1010: display_out = 7'b1110111; // A
29         4'b1011: display_out = 7'b0011111; // b
30         4'b1100: display_out = 7'b1001110; // C
31         4'b1101: display_out = 7'b0111101; // d
32         4'b1110: display_out = 7'b1001111; // E
33         4'b1111: display_out = 7'b1000111; // F
34     default: display_out = 7'b0000000; // All off for safety
35 endcase
36 end
37
38 // Assign individual segment outputs
39 {a, b, c, d, e, f, g} = display_out;
40
41 end
42
43
44 endmodule

```

6.4 Testbench

The following testbench was used to verify the functionality of the RTL design using different input combinations.

```

1  `timescale 1ns/1ps
2
3  module segment_LED_Display_tb;
4      reg [3:0] A, B;
5      reg [1:0] opcode;
6      reg      enable;
7      wire     a, b, c, d, e, f, g;
8
9      segment_LED_Display DUT (
10         .A(A), .B(B), .opcode(opcode), .enable(enable),
11         .a(a), .b(b), .c(c), .d(d), .e(e), .f(f), .g(g)
12     );
13
14     initial begin
15         $monitor("time=%0t en=%b A=%h B=%h op=%b | abcdefg=%b",

```

```

16         $time, enable, A, B, opcode, {a,b,c,d,e,f,g});
17
18     enable = 1;
19     A = 4'd3; B = 4'd4; opcode = 2'b00; #10; // 3 + 4 = 7
20     A = 4'd5; B = 4'd2; opcode = 2'b01; #10; // 5 / 2 = 2
21     A = 4'd9; B = 4'd1; opcode = 2'b10; #10; // 9 - 1 = 8
22     A = 4'hF; B = 4'h5; opcode = 2'b11; #10; // F ^ 5 = A
23     A = 4'd6; B = 4'd6; opcode = 2'b00; #10; // 6 + 6 = C
24     enable = 0;                               #10; // display off
25
26     $stop;
27 end
28 endmodule

```

6.5 Simulation Waveform

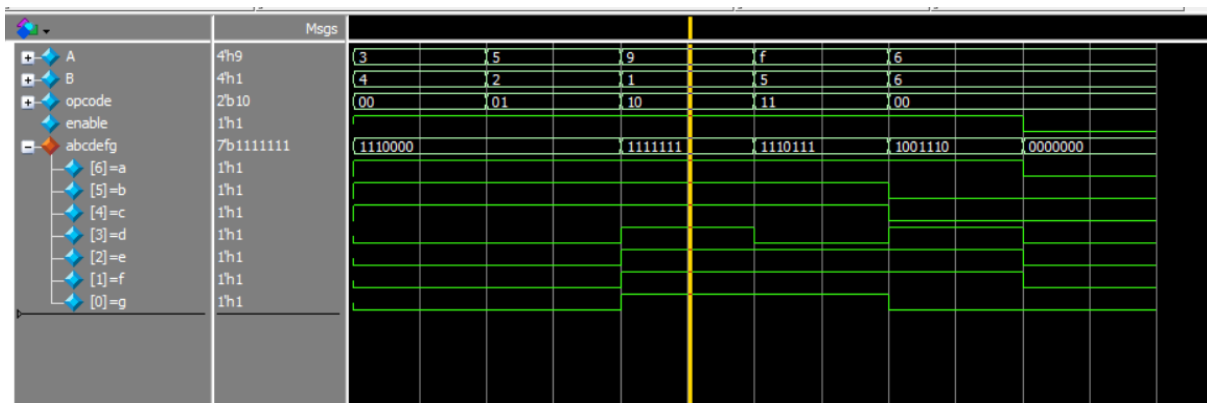


Figure 9: Simulation waveform for Question 6

```

# Errors: 0, Warnings: 0
# vsim -voptargs="+acc" work.segment_LED_Display_tb
# Start time: 01:00:30 on Sep 22, 2026
# ** Note: (vsim-3812) Design is being optimized...
# Loading work.segment_LED_Display_tb(fast)
# Loading work.segment_LED_Display(fast)
# Loading work.Nbit_ALU(fast)
# time=0 en=1 A=3 B=4 op=00 | abcdefg=1110000
# time=10000 en=1 A=5 B=2 op=01 | abcdefg=1110000
# time=20000 en=1 A=9 B=1 op=10 | abcdefg=1111111
# time=30000 en=1 A=f B=5 op=11 | abcdefg=1110111
# time=40000 en=1 A=6 B=6 op=00 | abcdefg=1001110
# time=50000 en=0 A=6 B=6 op=00 | abcdefg=0000000
# ** Note: $stop : segment_LED_Display_tb.v(26)
# Time: 60 ns Iteration: 0 Instance: /segment_LED_Display_tb
# Break in Module segment_LED_Display_tb at segment_LED_Display_tb.v line 26

```

Figure 10: Transcript for Question 6

7 Conclusion

All circuit designs were implemented in Verilog and simulated using QuestaSim. The simulation waveforms were used to verify the expected functionality of each design.